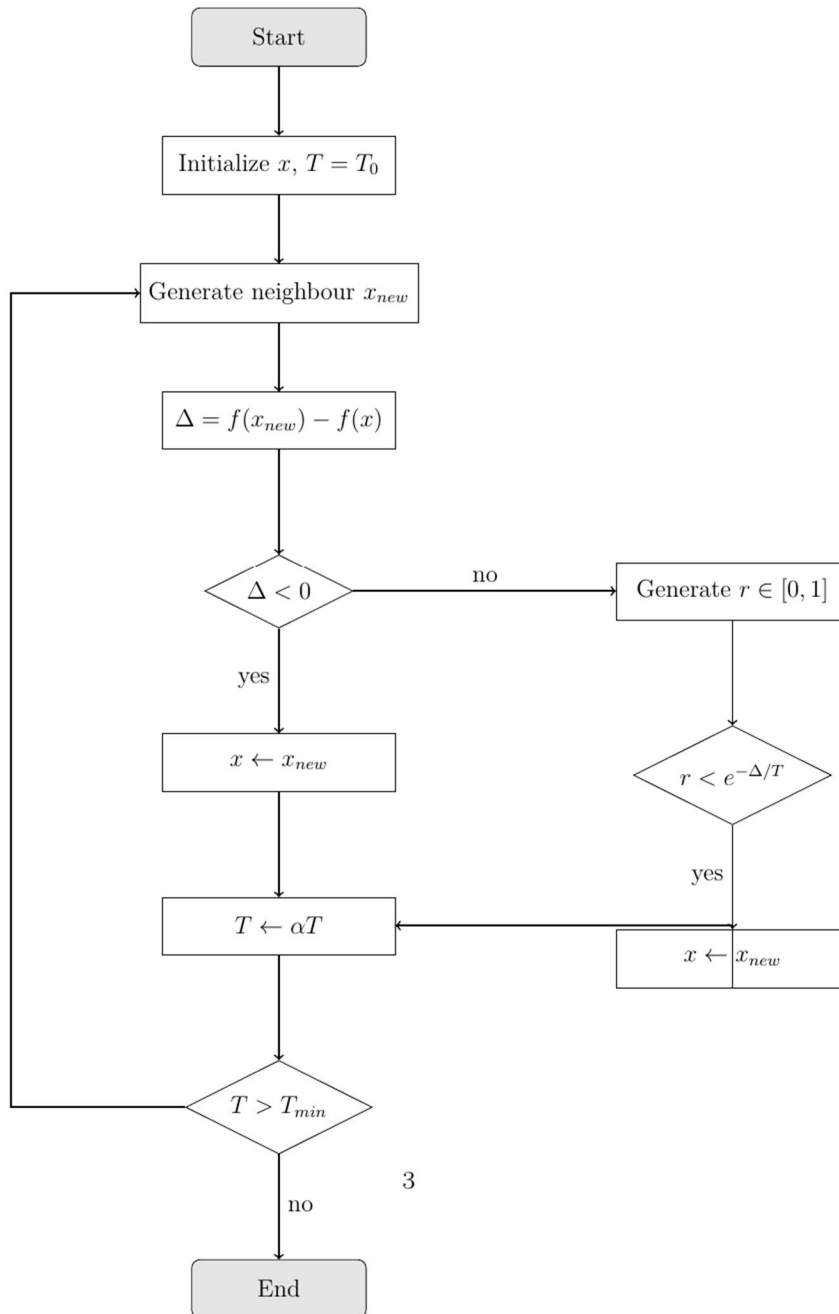


Algorytm wyżarzania symulowanego (Simulated Annealing)

Wprowadzenie

Algorytm wyżarzania symulowanego (Simulated Annealing) jest metodą optymalizacji inspirowaną procesem fizycznego wyżarzania metali. W metalurgii materiał jest podgrzewany do wysokiej temperatury, a następnie powoli chłodzony, co pozwala atomom osiągnąć bardziej stabilne energetycznie ułożenie.

W informatyce analogiczny proces stosuje się do wyszukiwania dobrego rozwiązania problemów optymalizacyjnych. Algorytm stopniowo poprawia rozwiązanie, ale czasami akceptuje również gorsze rozwiązania, aby uniknąć utknięcia w minimum lokalnym.



Rysunek 1: Flowchart of the simulated annealing algorithm

Idea działania

Algorytm rozpoczyna od rozwiązania początkowego i temperatury początkowej. W kolejnych krokach generowane są nowe rozwiązania z sąsiedztwa aktualnego rozwiązania.

Jeżeli nowe rozwiązanie jest lepsze, zostaje przyjęte. Jeżeli jest gorsze, może zostać zaakceptowane z pewnym prawdopodobieństwem zależnym od temperatury. Wraz z

kolejnymi iteracjami temperatura maleje, a algorytm staje się bardziej zachłanny i akceptuje głównie poprawy rozwiązania.

Elementy algorytmu

1. Funkcja celu – funkcja opisująca jakość rozwiązania.
2. Rozwiązanie początkowe – punkt startowy algorytmu.
3. Sąsiedztwo rozwiązania – sposób generowania nowych rozwiązań.
4. Temperatura – parametr kontrolujący losowość procesu.
5. Harmonogram chłodzenia – sposób zmniejszania temperatury.

Reguła akceptacji

Jeżeli nowe rozwiązanie jest lepsze, jest zawsze przyjmowane.
Jeżeli jest gorsze, może zostać zaakceptowane z prawdopodobieństwem:

$$P = \exp(-\Delta E / T)$$

gdzie:

ΔE – różnica wartości funkcji celu

T – aktualna temperatura

Pseudokod algorytmu

Algorithm 1: Simulated Annealing

Input: Objective function $f(x)$, initial solution x , initial temperature T_0

Output: Best solution x_{best}

$x_{curr} \leftarrow x;$

$x_{best} \leftarrow x;$

$T \leftarrow T_0;$

while $T > T_{min}$ **do**

for $i = 1$ **to** L **do**

$x_{new} \leftarrow \text{GenerateNeighbour}(x_{curr});$

$\Delta \leftarrow f(x_{new}) - f(x_{curr});$

if $\Delta < 0$ **then**

$x_{curr} \leftarrow x_{new};$

else

 generate $r \in [0, 1];$

if $r < e^{-\Delta/T}$ **then**

$x_{curr} \leftarrow x_{new};$

if $f(x_{curr}) < f(x_{best})$ **then**

$x_{best} \leftarrow x_{curr};$

$T \leftarrow \alpha T;$

return x_{best}

- x – initial solution
- x_{curr} – current solution
- x_{best} – best solution found during the search
- x_{new} – newly generated neighbouring solution
- $f(x)$ – objective function
- Δ – change in the objective function value
- T – current temperature
- T_0 – initial temperature
- T_{min} – minimum temperature (stopping condition)
- L – number of iterations performed at a given temperature
- α – cooling coefficient ($0 < \alpha < 1$)
- r – random number drawn from the interval $[0, 1]$

Harmonogram chłodzenia

Najczęściej stosuje się chłodzenie geometryczne:

$$T(k + 1) = \alpha * T(k)$$

gdzie współczynnik α mieści się zwykle w zakresie 0.8 – 0.99.

Zastosowania

Algorytm wyżarzania symulowanego stosuje się m.in. w:

- problemie komiwojażera,
- planowaniu tras dronów,
- optymalizacji harmonogramów,
- projektowaniu układów scalonych,
- uczeniu maszynowym.

Podsumowanie

Algorytm wyżarzania symulowanego jest skuteczną metodą optymalizacji dla złożonych problemów. Dzięki kontrolowanej losowości i stopniowemu zmniejszaniu temperatury algorytm może uniknąć minimów lokalnych i znaleźć rozwiązania bliskie optimum globalnego.

Przykład 1

```
import math
import random
from typing import Callable, Tuple

def objective(x: float) -> float:
    # Przykładowa funkcja celu do minimalizacji
    return x**2 + 10 * math.sin(x)

def generate_neighbour(x: float, step: float, lower: float,
upper: float) -> float:
    x_new = x + random.uniform(-step, step)
    return max(lower, min(upper, x_new))

def simulated_annealing(
    func: Callable[[float], float],
    x0: float,
    lower: float,
    upper: float,
    t0: float = 100.0,
    t_min: float = 1e-3,
    alpha: float = 0.95,
    iterations_per_temp: int = 100,
    step: float = 0.5,
) -> Tuple[float, float]:
    x_curr = x0
    x_best = x0
    f_curr = func(x_curr)
    f_best = f_curr
    t = t0

    while t > t_min:
        for _ in range(iterations_per_temp):
            x_new = generate_neighbour(x_curr, step, lower,
upper)
            f_new = func(x_new)
            delta = f_new - f_curr

            if delta < 0 or random.random() < math.exp(-delta /
t):
                x_curr = x_new
                f_curr = f_new
```

```
        if f_curr < f_best:
            x_best = x_curr
            f_best = f_curr

    t *= alpha

    return x_best, f_best

if __name__ == "__main__":
    random.seed(42)

    best_x, best_f = simulated_annealing(
        func=objective,
        x0=5.0,
        lower=-10.0,
        upper=10.0,
        t0=100.0,
        t_min=0.001,
        alpha=0.95,
        iterations_per_temp=100,
        step=0.5,
    )

    print(f"Najlepsze x: {best_x:.6f}")
    print(f"Minimum funkcji: {best_f:.6f}")
```